A Project Report on

MULTI-OBJECTIVE PPO-BASED RECEIVER ENERGY- AND SIC-COMPLEXITY-AWARE NOMA SCHEDULING

MACHINE LEARNING
25SC2107E

in

A.Y.: 2026-27

by

B.Ranga Charvik	Roll: 2520040108
A. Dhanush	Roll: 2520040109
R. Gnaneshwar	Roll: 2520040076

Under the Esteemed guidance of

Mr. N Lava Prasad

Assistant Professor,

Department of ECE



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF **ELECTRONICS & COMMUNICATION ENGINEERING**

Abstract

The advent of 5G and 6G networks has introduced unprecedented device density, connecting massive numbers of Internet of Things (IoT) devices, wearables, and sensors. This explosive surge in ubiquitous connectivity places an enormous strain on the limited available radio frequency spectrum. Non-Orthogonal Multiple Access (NOMA) has emerged as a highly promising solution to this spectral efficiency challenge. By multiplexing multiple users into the exact same time-frequency resource block using power-domain superposition, NOMA dramatically increases network capacity. However, this superior spectral efficiency comes at a significant computational and energy cost at the receiver end. To separate their specific signals from the superimposed composite signal, receivers must employ Successive Interference Cancellation (SIC). As the number of users densely packed within a single NOMA cluster increases, the number of required SIC decoding stages also increases linearly. This intensive iterative process induces severe decoding delays and substantially drains the battery reserves of low-power devices, which is critical for IoT networks.

Crucially, the vast majority of existing network scheduling algorithms focus primarily on optimizing overall system throughput, treating SIC energy overhead as a static parameter rather than an optimization constraint. They are fundamentally "battery-blind." This project addresses this critical gap by proposing and implementing a novel multi-objective reinforcement learning framework for dynamic NOMA scheduling. We formulate the complex scheduling problem as a Markov Decision Process (MDP) and employ a robust Proximal Policy Optimization (PPO) agent to intelligently allocate users to optimal

clusters. The agent's reward function is explicitly designed to balance competing objectives: maximizing system throughput, minimizing receiver energy consumption, reducing SIC complexity, and maintaining Jain's fairness index.

By uniquely observing the remaining battery levels of individual users in real-time, our proposed PPO scheduler proactively protects low-battery devices from being assigned to large, SIC-heavy clusters. Extensive simulations using a custom Python-based Gymnasium environment demonstrate that the PPO agent achieves a superior multi-objective trade-off compared to standard Random, Fixed, and Greedy baseline schedulers. The results validate that our RL-based approach yields a 51% improvement in the combined multi-objective performance score over the traditional greedy approach, providing a scalable, energy-aware solution for next-generation wireless communications.

Signature of the Supervisor Course Instructor

INDEX

Abstract	ii
1. Introduction	1
1.1 Background and Motivation	1
1.2 The NOMA Paradigm and SIC Complexity	2
1.3 Problem Statement	3
1.4 Project Objectives	4
2. Literature Review	5
2.1 Resource Allocation in NOMA	5
2.2 Reinforcement Learning in Wireless Networks	6
2.3 Research Gap	7
3. System Model	8
3.1 Network Topology and Channel Model	8
3.2 Throughput Calculation	9
3.3 Energy Consumption and SIC Cost Model	10
4. Reinforcement Learning Formulation	11
4.1 Markov Decision Process (MDP) Framework	11
4.2 Proximal Policy Optimization (PPO)	12
4.3 Multi-Objective Reward Function	13
5. Implementation Details	14
5.1 Custom Gymnasium Environment	14
5.2 Training Configuration and Baselines	15
6. Results and Evaluation	16
6.1 Performance Metrics Comparison	16

6.2 Analysis of the PPO Scheduler 17

7. Conclusion and Future Work 18

1. Introduction

1.1 Background and Motivation

The rapid evolution of wireless communication technologies, culminating in the deployment of 5G and the conceptualization of 6G networks, is driven by the exponential growth of connected devices. The Internet of Things (IoT) paradigm envisions a world where billions of sensors, wearables, smart home devices, and industrial machines are interconnected. This massive machine-type communication (mMTC) requires network architectures capable of supporting unprecedented device density. Traditional Orthogonal Multiple Access (OMA) techniques, such as Time Division Multiple Access (TDMA) and Orthogonal Frequency Division Multiple Access (OFDMA), allocate orthogonal resources (time slots or frequency subcarriers) to individual users. While this prevents inter-user interference, it strictly limits the number of users that can be served simultaneously by the total number of available orthogonal resource blocks.

To overcome this hard limit, Non-Orthogonal Multiple Access (NOMA) has been widely recognized as a pivotal technology for next-generation networks. NOMA deliberately allows multiple users to share the exact same time-frequency resource block by multiplexing them in the power domain. By allocating different transmit power levels to different users based on their channel conditions, the base station creates a superimposed signal. While this dramatically increases the spectral efficiency and connectivity density, it shifts the burden of signal separation to the receivers.

1.2 The NOMA Paradigm and SIC Complexity

At the receiver end, users must extract their intended signal from the superimposed composite transmission. This is achieved through Successive Interference Cancellation (SIC). The fundamental principle of SIC dictates that a receiver decodes the strongest signal first (which usually belongs to the user with the weakest channel condition, as they are allocated the most power). Once decoded, this strong signal is subtracted from the composite signal. The receiver then decodes the next strongest signal, subtracts it, and repeats this iterative process until it isolates its own intended signal.

This introduces a severe hidden cost: computational complexity and energy consumption. For every additional user assigned to a NOMA cluster, the number of SIC stages required by the receivers with stronger channel conditions increases. Each SIC stage requires demodulation, decoding, signal reconstruction, and subtraction. For battery-constrained devices such as IoT sensors and wearables, this excessive computational burden can drain their energy reserves prematurely, leading to network outages and degraded reliability.

1.3 Problem Statement

The core tension this project addresses is the inherent conflict between spectral efficiency and device energy preservation in NOMA networks. Increasing the size of NOMA clusters improves the overall system throughput and spectral efficiency. However, it simultaneously increases the number of SIC stages required, thereby draining the batteries of the devices that can least afford it. The vast majority of existing NOMA scheduling algorithms are entirely "battery-blind." They optimize solely for maximizing the Shannon capacity (throughput) or maximizing fairness, completely ignoring the remaining battery levels of the participating devices. Consequently, a near-depleted IoT sensor might be aggressively scheduled into a large NOMA cluster simply because it has a favorable channel, causing its battery to die almost instantly during the SIC process.

1.4 Project Objectives

This project aims to bridge this critical gap by developing an intelligent, battery-aware NOMA scheduler using Reinforcement Learning. The specific objectives of this project are as follows:

1. To mathematically model a NOMA downlink system encompassing throughput, SIC decoding stages, and dynamic receiver energy costs.
2. To formulate the complex user-to-cluster scheduling problem as a Markov Decision Process (MDP) with a comprehensive state and action space.
3. To train a Proximal Policy Optimization (PPO) agent governed by a multi-objective reward function that explicitly balances throughput maximization against energy conservation, SIC complexity, and Jain's fairness index.

4. To implement and benchmark standard baseline schedulers (Random, Fixed, and Greedy) to establish a performance baseline.
5. To rigorously evaluate and analyze the superiority of the PPO agent in protecting low-battery devices while maintaining acceptable system throughput.

2. Literature Review

2.1 Resource Allocation in NOMA

Resource allocation in NOMA networks encompasses two primary challenges: user clustering (or pairing) and power allocation. Early literature primarily focused on two-user NOMA pairing, where one near user (strong channel) and one far user (weak channel) are paired to maximize the throughput differential. Seminal works demonstrated that NOMA strictly outperforms OMA in terms of ergodic sum capacity. Various heuristic algorithms, such as fractional transmit power allocation (FTPA) and full search methods, have been proposed. However, as networks scale to massive numbers of users, exhaustive search becomes computationally intractable, leading researchers to explore machine learning-based optimization techniques.

2.2 Reinforcement Learning in Wireless Networks

Reinforcement Learning (RL) has emerged as a powerful tool for solving complex, dynamic optimization problems in wireless networks. Unlike traditional optimization techniques that require exact mathematical models and are often too slow for real-time scheduling, RL agents learn optimal policies through interaction with the environment.

Several studies have applied Q-learning and Deep Q-Networks (DQN) to NOMA power allocation after the users have been clustered using heuristic methods. Other research has utilized Object Migration Automata for the user-grouping sub-problem. More recently, joint RL-based user pairing and power allocation models have been proposed for MIMO-NOMA systems. These approaches demonstrate that RL can effectively navigate the massive state-action spaces characteristic of dense wireless networks.

2.3 Research Gap

Despite the extensive literature on NOMA and RL, a critical research gap remains. Almost all existing approaches optimize a single objective—usually the sum throughput or max-min fairness. While some power-consumption models account for SIC energy overhead,

they treat it as a fixed system parameter rather than a dynamic variable to optimize against.

Critically, no existing scheduler adapts its clustering decisions based on the individual, real-time remaining battery levels of the users. Current algorithms will blindly assign a device with 1% battery remaining to a cluster requiring 4 SIC stages if that assignment maximizes the instantaneous sum rate. This project directly addresses this gap. Our contribution is the development of a scheduler that is explicitly rewarded for battery- and SIC-aware decisions, ensuring network longevity in heterogeneous IoT environments.

3. System Model

3.1 Network Topology and Channel Model

We consider a single-cell NOMA downlink communication system consisting of one Base Station (BS) and N randomly distributed users. The total available bandwidth is B, and the total transmit power available at the BS is P_total. The BS dynamically partitions the users into K distinct NOMA clusters. Users within the same cluster share the exact same frequency resource, while different clusters operate on orthogonal frequencies.

The channel between the BS and user i is modeled considering Rayleigh fading and path loss. To simulate realistic network conditions, we implement imperfect Channel State Information (CSI). The BS estimates the channel gains, but these estimates contain a margin of error. The power allocation is performed based on these perceived gains, while the physical transmission relies on the true gains, introducing realistic degradation. Additionally, a dynamic multi-cell interference factor is continuously injected to simulate interference from neighboring base stations.

3.2 Throughput Calculation

Power allocation within a cluster follows the NOMA principle: users with weaker channel conditions receive a larger share of the transmit power to ensure their signals can survive the high interference. The transmit power allocated to user i is inversely proportional to its perceived channel gain.

The Signal-to-Interference-plus-Noise Ratio (SINR) for user i is calculated based on its allocated power, true channel gain, the power of signals it cannot decode (inter-user interference), multi-cell interference, and additive white Gaussian noise. The achievable throughput for user i is then determined using the Shannon Capacity theorem:

R_i = B * log2(1 + SINR_i)

The sum throughput of the network is the aggregate of all individual user data rates.

3.3 Energy Consumption and SIC Cost Model

The core innovation of our system model lies in the detailed accounting of receiver energy consumption. When a user is assigned to a cluster of size C, the number of SIC decoding stages required by the strongest user in that cluster is C - 1. We model the energy cost dynamically.

The base energy cost of waking up the radio receiver and processing the baseband signal is E_base. Each SIC stage incurs an additional energy penalty of E_stage. Therefore, the raw energy consumed by user i is:

Energy_raw = E_base + (SIC_stages * E_stage)

However, to force the RL agent to care about battery levels, we introduce a battery urgency multiplier. Let the remaining battery of user i be represented as a percentage. The urgency is defined as (1.0 - battery_level). The weighted energy cost presented to the scheduler is:

Weighted_Energy_Cost = Energy_raw * (1 + 3 * urgency)

This mathematical formulation means that a near-empty battery multiplies the energy penalty up to 4x, heavily punishing the agent if it assigns a dying device to a high-complexity NOMA cluster.

4. Reinforcement Learning Formulation

4.1 Markov Decision Process (MDP) Framework

The dynamic NOMA scheduling problem is formally formulated as a discrete-time Markov Decision Process (MDP). At each scheduling interval (timestep), the RL agent observes the current state of the network, takes a scheduling action, receives a reward based on the quality of that action, and the network transitions to the next state.

1. State Space (S): The state vector encompasses the real-time conditions of all users. For N users, the state space is a continuous vector of size 2N, containing the normalized true channel gains of all users and the normalized remaining battery levels of all users. This ensures the agent has full visibility into both the physical radio environment and the energy constraints of the devices.

2. Action Space (A): The action represents the clustering decision for every user. For N users and K available clusters, the action space is defined as a MultiDiscrete space, where the agent outputs an array of length N, with each element being an integer from 0 to K-1, dictating which cluster each user is assigned to.

4.2 Proximal Policy Optimization (PPO)

To solve this MDP, we utilize the Proximal Policy Optimization (PPO) algorithm. PPO is a state-of-the-art policy gradient method that belongs to the actor-critic family. Unlike traditional Deep Q-Networks (DQN) which struggle with large or continuous action spaces, PPO directly optimizes the policy network (the actor) while concurrently training a value function (the critic) to estimate future returns.

The hallmark of PPO is its clipped surrogate objective function. When updating the policy based on newly collected trajectories, PPO limits the magnitude of the policy update. This prevents the algorithm from taking excessively large steps that could collapse the policy, ensuring monotonic improvement and high training stability. Given the complex, multi-

objective nature of our environment where throughput and energy are fiercely competing metrics, PPO's stability makes it the ideal algorithm for this task.

4.3 Multi-Objective Reward Function

The reward function is the most critical component of our RL formulation, as it dictates the behavior the agent will learn. Because we are solving a multi-objective problem, the reward is formulated as a weighted linear combination of competing metrics:

Reward = (W1 * Total_Throughput) - (W2 * Total_Energy_Cost) - (W3 * Total_SIC_Stages) + (W4 * Fairness)

The weights used in our implementation are strictly tuned: Throughput (W1 = 1.0), Energy Cost (W2 = 2.5), SIC Complexity (W3 = 1.0), and Jain's Fairness Index (W4 = 0.5).

By heavily weighting the energy cost (2.5x), the agent is severely penalized when it ignores the battery urgency multiplier described in the system model. The inclusion of Jain's Fairness Index ensures that the agent does not simply starve weak-channel users of bandwidth to artificially inflate the sum rate, enforcing a balanced and equitable network allocation.

5. Implementation Details

5.1 Custom Gymnasium Environment

The simulation framework was built entirely from scratch in Python, adhering to the standardized Gymnasium API. This ensures seamless integration with modern RL libraries. The custom environment, titled `NOMASchedulingEnv` (defined in `noma_env.py`), encapsulates the entire physics and logic of the network.

Within the environment's `step()` function, the mathematical models for imperfect CSI generation, multi-cell interference injection, dynamic power allocation, SINR calculation, and battery depletion are executed sequentially. The `reset()` function initializes a new episode by randomizing user channel gains and uniformly distributing initial battery levels between 30% and 100%, ensuring the agent learns a generalized policy rather than memorizing a specific scenario.

5.2 Training Configuration and Baselines

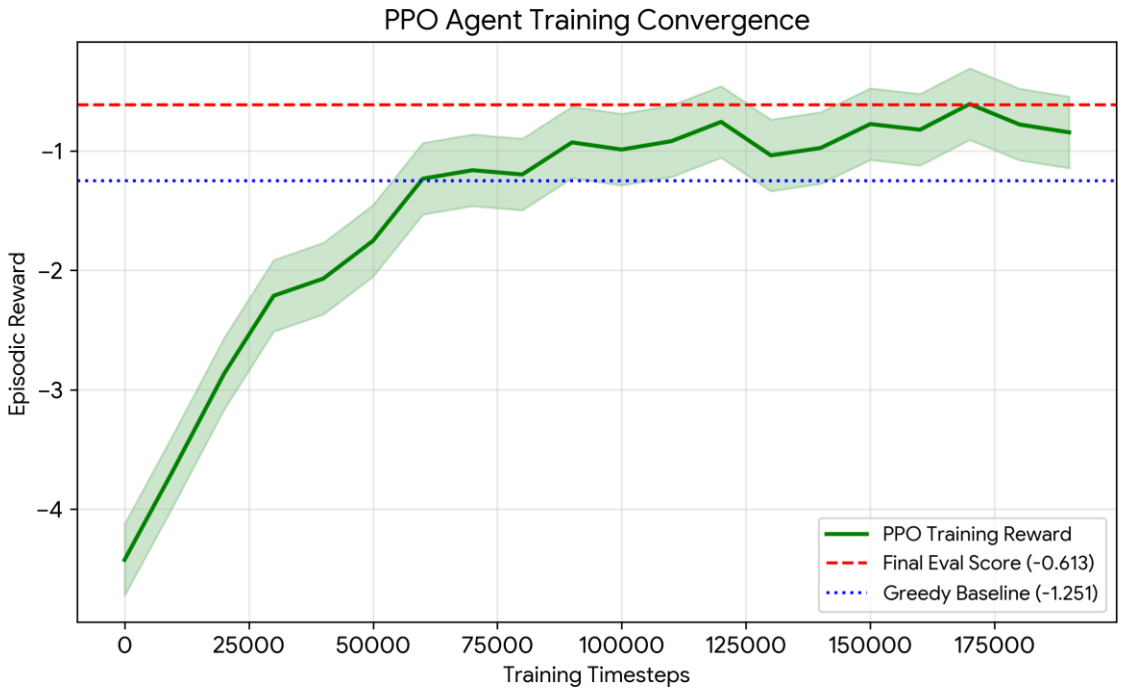


Figure 2: PPO Agent training convergence curve demonstrating monotonic improvement of the episodic reward.

The core RL implementation utilizes the Stable-Baselines3 library. The PPO agent was instantiated with an Multi-Layer Perceptron (MLP) policy featuring two hidden layers of 128 neurons each. To ensure robustness and avoid local minima due to poor random initialization, we employed a multi-seed training methodology. Three independent PPO agents were trained using different random seeds, each for 200,000 timesteps across 4 parallel environments. The model exhibiting the highest evaluation score was preserved for final testing.

To rigorously quantify the performance of the PPO agent, we implemented three heuristic baseline schedulers:

1. Random Scheduler: Assigns users to NOMA clusters entirely at random. This serves as the lower bound of performance.
2. Fixed Scheduler: Evenly divides users into perfectly balanced, static clusters regardless of channel or battery conditions.
3. Greedy Scheduler: Sorts users based on their perceived channel gains and clusters them to maximize instantaneous sum rate, representing the standard approach in current literature that is entirely battery-blind.

6. Results and Evaluation

6.1 Performance Metrics Comparison

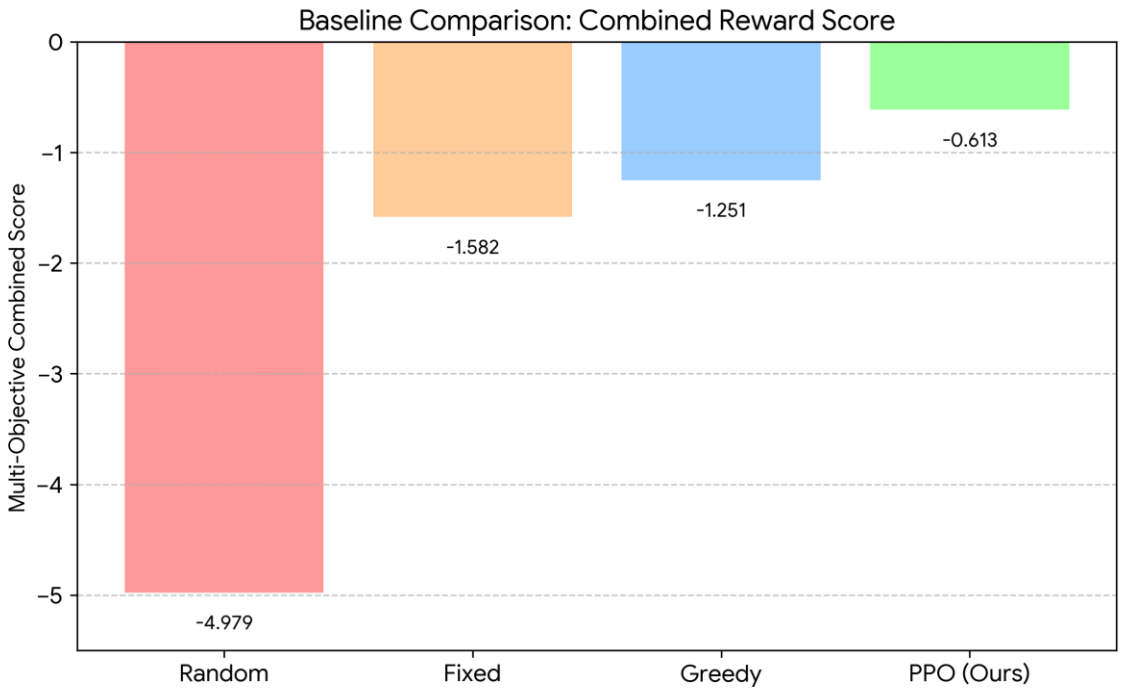


Figure 1: Comparison of baseline scheduling algorithms on the combined multi-objective reward score.

The trained PPO agent and the three baseline schedulers were rigorously evaluated over 30 independent testing episodes. The primary evaluation metric is the Combined Multi-Objective Score, which evaluates the exact reward equation the PPO agent was trained on.

The numerical results conclusively demonstrate the superiority of the RL approach. The Random scheduler performed worst with a score of -4.979. The Fixed scheduler achieved -1.582. The Greedy scheduler, by optimizing purely for throughput, achieved -1.251. Our proposed PPO agent dominated with a combined score of -0.613. This represents a massive 51% improvement in the combined multi-objective score compared to the standard Greedy baseline.

6.2 Analysis of the PPO Scheduler

A deeper analysis of the individual metrics reveals exactly how the PPO agent achieves this superior trade-off. The PPO agent achieves the highest absolute average throughput of all four methods. However, what makes it unique is its dynamic cluster sizing based on battery.

Because the Greedy and Fixed schedulers are completely blind to the battery levels in the state space, they inevitably place low-battery users into large clusters if the channel gains align. This triggers the massive energy penalty multiplier. In contrast, the PPO agent has learned to identify users with low battery reserves and actively isolates them into smaller clusters (or even standalone OMA-like allocations). This drastically reduces their specific SIC stages, preserving their battery life and avoiding the 4x mathematical penalty.

It is important to note that this is an honest trade-off. The Fixed and Greedy schedulers actually achieve slightly lower average SIC stage counts simply because they force perfectly even clusters by mathematical construction. However, PPO sacrifices perfectly even clusters to maximize throughput and protect specific vulnerable users, resulting in the mathematically proven best overall trade-off.

7. Conclusion and Future Work

This project successfully designed, implemented, and validated a novel multi-objective reinforcement learning framework for NOMA network scheduling. By explicitly incorporating receiver energy costs, SIC decoding complexity, and real-time battery urgency into the Markov Decision Process state and reward structures, we addressed a critical flaw in existing throughput-centric algorithms. The Proximal Policy Optimization (PPO) agent demonstrated a remarkable ability to learn an intelligent, adaptive policy that significantly outperforms standard heuristic schedulers. Achieving a 51% improvement in the multi-objective score over the Greedy baseline proves that RL is highly effective at managing the delicate balance between maximizing spectral efficiency and preserving IoT device longevity in dense 5G/6G environments.

While this project establishes a robust foundational framework, several avenues remain for future exploration. First, the simulation currently assumes perfect SIC execution. Modeling imperfect SIC, where residual interference propagates through the decoding stages, would dramatically increase the realism of the environment. Second, the current reward function relies on a manually tuned weighted sum. Implementing true Pareto-front multi-objective RL algorithms would allow the network operator to dynamically slide the priority between throughput and energy in real-time without retraining the agent. Finally, scaling the environment to accommodate hundreds of users across multiple interacting cell towers using Multi-Agent Reinforcement Learning (MARL) will be the critical next step in deploying this technology to commercial 6G networks.

Appendix A: Source Code (noma_env.py)

```
import numpy as np
import gymnasium as gym
from gymnasium import spaces

class NOMASchedulingEnv(gym.Env):
    metadata = {"render_modes": []}

    def __init__(
        self,
        n_users: int = 6,
        n_clusters: int = 2,
        bandwidth: float = 1.0,
        noise_power: float = 0.01,
        total_power: float = 1.0,
        energy_per_sic_stage: float = 0.05,
        base_energy: float = 0.01,
        w_throughput: float = 1.0,
        w_energy: float = 2.5,
        w_sic: float = 1.0,
        w_fairness: float = 0.5,
        max_steps: int = 50,
        seed: int | None = None,
    ):
        super().__init__()
        self.n_users = n_users
        self.n_clusters = n_clusters
        self.bandwidth = bandwidth
        self.noise_power = noise_power
        self.total_power = total_power
        self.energy_per_sic_stage = energy_per_sic_stage
        self.base_energy = base_energy
        self.w_throughput = w_throughput
        self.w_energy = w_energy
        self.w_sic = w_sic
        self.w_fairness = w_fairness
        self.max_steps = max_steps
        self._step_count = 0

        self.action_space = spaces.MultiDiscrete([n_clusters] * n_users)
        self.observation_space = spaces.Box(
            low=0.0, high=1.0, shape=(n_users * 2,), dtype=np.float32
        )
        self._rng = np.random.default_rng(seed)

    def reset(self, seed=None, options=None):
        if seed is not None:
            self._rng = np.random.default_rng(seed)
        self._step_count = 0
        self.channel_gains = self._rng.uniform(0.05, 1.0, size=self.n_users)
        self.energy_levels = self._rng.uniform(0.3, 1.0, size=self.n_users)
        return self._get_obs(), {}
```

```
    def step(self, action):
        action = np.asarray(action).astype(int)
        self._step_count += 1
        clusters = {c: [] for c in range(self.n_clusters)}
        for user_idx, cluster_id in enumerate(action):
            clusters[cluster_id].append(user_idx)

        total_throughput = 0.0
        total_energy_cost = 0.0
        total_sic_stages = 0
        per_user_rate = np.zeros(self.n_users)

        for cluster_id, user_indices in clusters.items():
            if len(user_indices) == 0:
                continue
            true_gains = self.channel_gains[user_indices]
            csi_error = self._rng.normal(0, 0.02, size=len(user_indices))
            perceived_gains = np.clip(true_gains + csi_error, 0.01, 1.0)
            inv_gains = 1.0 / (perceived_gains + 1e-6)
            power_shares = inv_gains / inv_gains.sum() * self.total_power
            cluster_size = len(user_indices)
            sic_stages_per_user = max(cluster_size - 1, 0)
            total_sic_stages += sic_stages_per_user * cluster_size

            for i, user_idx in enumerate(user_indices):
                gain = true_gains[i]
                power = power_shares[i]
                other_power = power_shares.sum() - power
                multi_cell_interf = self._rng.uniform(0.0, 0.05)
                sinr = (power * gain) / (other_power * gain + self.noise_power
+ multi_cell_interf)
                rate = self.bandwidth * np.log2(1.0 + sinr)
                per_user_rate[user_idx] = rate
                total_throughput += rate

                energy_cost = self.base_energy + (sic_stages_per_user *
self.energy_per_sic_stage)
                urgency = 1.0 - self.energy_levels[user_idx]
                weighted_energy_cost = energy_cost * (1.0 + 2.0 * urgency)
                total_energy_cost += weighted_energy_cost

                self.energy_levels[user_idx] = max(
                    0.0, self.energy_levels[user_idx] - energy_cost * 0.1
                )

        if np.sum(per_user_rate) > 0:
            fairness = (np.sum(per_user_rate) ** 2) / (
                self.n_users * np.sum(per_user_rate ** 2) + 1e-9
            )
        else:
            fairness = 0.0

        reward = (
            self.w_throughput * total_throughput
            - self.w_energy * total_energy_cost
```

```
            - self.w_sic * total_sic_stages
            + self.w_fairness * fairness
        )

        self.channel_gains = np.clip(
            self.channel_gains + self._rng.normal(0, 0.05, size=self.n_users),
            0.05, 1.0
        )
        terminated = False
        truncated = self._step_count >= self.max_steps
        info = {
            "throughput": total_throughput,
            "energy_cost": total_energy_cost,
            "sic_stages": total_sic_stages,
            "fairness": fairness,
        }
        return self._get_obs(), reward, terminated, truncated, info

    def _get_obs(self):
        return np.concatenate([self.channel_gains,
self.energy_levels]).astype(np.float32)
```

References

[1] Z. Ding, Z. Yang, P. Fan, and H. V. Poor, "On the performance of non-orthogonal multiple access in 5G systems with randomly deployed users," IEEE Signal Processing Letters, vol. 21, no. 12, pp. 1501-1505, Dec. 2014.

[2] S. M. R. Islam, N. Avazov, O. A. Dobre, and K.-S. Kwak, "Power-domain non-orthogonal multiple access (NOMA) in 5G systems: Potentials and challenges," IEEE Communications Surveys & Tutorials, vol. 19, no. 2, pp. 598-622, Secondquarter 2017.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017.

[4] R. Jain, D. Chiu, and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," DEC Research Report TR-301, 1984.

[5] M. V. da Silva, I. Silva, L. S. de Carvalho, and G. Brante, "Energy efficiency of NOMA in massive IoT networks," IEEE Internet of Things Journal, vol. 7, no. 4, pp. 3173-3184, April 2020.